EECE 7205 — Final Project Proposal

Academic Advising and Course Planning Tool

MyungJin Suh | Dr. Fatema Nafa | Fall 2026 | October 26, 2026

Individual project, one member. Topic from the Iteration 1 recommended list. A menu-driven C++ program builds a prerequisite graph for the Northeastern BSCmpE and writes a semester plan. It compares topological earliest-semester packing with backtracking.

1 Project Title

Academic Advising and Course Planning Tool. Plans the remaining required courses for one BSCmpE student under prerequisite, term, and credit-cap constraints, and compares the two planners above.

2 Problem Statement and Proposed Solution

Problem. An advisor has to tell which requirements are done, which courses an OR-prerequisite and a "may be taken concurrently" note actually unlock, and which unlocked course should take the open seat when the cap, the offering term, and a preferred elective cannot all win. A local choice can add a semester.

User. An advisor, or a student preparing for that meeting. One degree, one synthetic student at a time. The program does not register anyone and does not store real names.

Why it matters. A plan that breaks a prerequisite cannot be followed. A prerequisite-correct plan can still waste a fall-only course. Those are different failures, so the project measures two algorithms.

What the program does. It loads the requirements, builds the clause graph, and stops if the edges cycle. Completed courses come off the graph. Both planners run. A separate `verifyPlan` checks strict prerequisites in an earlier semester, concurrent ones in the same semester or earlier, the offering term, and the credit cap. The screen shows each plan, its span (index of the last semester used, holes included), a preference penalty, and the time. Scope is the BSCmpE required list plus prerequisites those entries name, not the whole university, and not seat counts.

Features.

1. Load requirements and clauses transcribed from the catalog.
2. Load one synthetic student: completed courses, credit cap, preferred electives.
3. Build the graph and refuse a cycle.
4. Look up a course by code.
5. Plan by clause-count topological order and earliest-semester packing.
6. Plan by backtracking, pruning on the best span.
7. Verify both plans and print span, preference, and time.
8. Save the chosen plan.
9. Time the worst-case family on synthetic graphs.

Inputs, outputs, and sources. Requirements and prerequisite text, from the Northeastern undergraduate catalog:

https://catalog.northeastern.edu/undergraduate/engineering/

electrical-computer/computer-engineering-bscompe/

`https://catalog.northeastern.edu/course-descriptions/eece/` Three clauses are copied exactly because the tests use them. EECE 2160 (4 hours): EECE 2140 or CS 2510, either may be concurrent. EECE 2322 (4 hours): EECE 2160 or EECE 2310. EECE 2560 (4 hours): EECE 2160 or CS 1500. The file will hold every required course on the BSCmpE page and the prerequisites those rows name. Math, physics, and CS appear only when an EECE row lists them. Rows are transcribed, not scraped. If a catalog page does not state a term, the course is marked `FS` and the README says so.

Clauses are separated by `;`, AND-members by `+`, and a trailing `@` means concurrent. `offered` is `F`, `S`, or `FS`. A course unlocks when one clause is satisfied. A strict member must be earlier. A concurrent member may be earlier or in the same semester. Storing every alternative as a required edge would turn the OR into an AND.

```
code,title,credits,offered,clauses
EECE2560,Fundamentals of Engineering Algorithms,4,FS,EECE2160;CS1500

id,completed,maxCredits,prefer
S01,EECE2140,8,EECE2560
```

Student ids are `S01`, `S02`, ..., synthetic, not people. Saving writes `semester,code,title,credits`. Timing appends algorithm, V, median time, and whether the trial hit the cap.

Why this problem. It is the recommended advising topic, limited to one degree so one person can transcribe the data and still measure both planners. The work is a graph, a topological order, a backtracking search, and a best-case

and worst-case bound for each.

3 Team Structure, Contributions, and Objectives

Team. One member. The Iteration 1 sheet should use this same topic and description: MyungJin Suh; Academic Advising and Course Planning Tool; a C++ planner for the BSCmpE prerequisite graph, catalog rows transcribed from the undergraduate catalog, synthetic students, topological packing compared with backtracking, best- and worst-case time and space.

Objectives. A working planner under prerequisite, term, and cap constraints. The graph, both planners, the queue, and insertion sort are hand-written; a `vector` is only a resizable array while reading. Compare span, preference, and time. State best and worst time, and space, as Θ of the input that attains the bound. Test normal, boundary, and invalid cases, including a planted cycle. Use feature branches on `EECE7205_HW`.

Contributions — this iteration. Done: chose the topic and the BSCmpE bound; cited the catalog and checked the three clauses; defined the CSV, the OR rule, and the concurrent rule; specified both planners, the verifier, and the bounds; wrote the pseudocode, tests T1–T10, and the timing plan, including one input where the planners must disagree. Next, still mine: transcribe `data/catalog.csv`; implement load, the BST index, and cycle detection; implement both planners and the verifier and confirm T4; add the timing script. This iteration is the design. Programming and testing are the following iterations.

Learning objectives. Pass records by reference and const reference. Index codes with a BST and state the chain worst case. Implement an OR-clause topological order and a pruned backtracking search. Read and write CSV and reject a bad row. Derive the Θ bounds, then time the worst-case family. Use branches, small commits, and pull requests.

4 Program Design and GitHub Collaboration — This Iteration

4.1 Initial program design

The nine features above are the menu, choices 1–9.

Data structures. `Course` holds code, title, credits, fall, spring, and a required flag. `Clause` holds the owner index, up to four member indexes, a concurrent flag per member, and `left`, the members not yet completed. `Plan` holds `termOf[MAXC]` (-1 if unplaced), span, and preference (sum of term indexes of preferred electives). Catalog and clauses live in fixed arrays; overflow is an error. The catalog is a few dozen required courses.

A BST keyed by code maps to a catalog index. The node is the in-class shape: key, index, left, right. Lookup is $\Theta(1)$ at the root and $\Theta(V)$ on a chain. Sorted insertion builds that chain. The tree is not balanced. Space $\Theta(V)$. The graph is the clause array plus `int adj[MAXC][MAXD]`, the clauses that mention each course. Clause counts, not one indegree, keep an OR from becoming an AND. Space $\Theta(V + P)$, P the number of clause memberships. The Kahn queue is a ring buffer of indexes, not `std::queue`. A hand-written insertion sort orders a ready list by code: $\Theta(r)$ best, $\Theta(r^2)$ worst, $r \le V$.

Planners. Both place only unlocked courses and both refuse a cycle. `verifyPlan` does not share code with either search. Planner A builds the clause-count order (a course enters when some clause has `left = 0`, ties break toward the smaller code), then places each course in the earliest legal semester. That earliest fit is the greedy step, and T4 is where it picks the wrong fall-only course. Planner B tries semesters in that same order, earliest first, keeps the smallest span, and cuts a branch that cannot finish sooner. Preference breaks equal spans.

Bounds. V courses left to place, P clause memberships, S semesters ($S = 16$ in the program, eight years of fall and spring), r courses ready in one semester.

Operation	Time	Space
Load and build	$\Theta(V + P)$ best and worst	$\Theta(V + P)$
BST lookup	Best $\Theta(1)$, the root. Worst $\Theta(V)$, a chain	$\Theta(V)$
Topological order	$\Theta(V + P)$ best and worst. Each course is enqueued once; each membership is decremented once	$\Theta(V)$
Earliest-semester packing	$\Theta(VS + P)$ best and worst	$\Theta(V)$
Backtracking	Best $\Theta(V + P)$: a chain with one legal semester per course, and the span bound cuts the rest. Worst $\Theta(S^V)$: $P = 0$, every course offered every term, cap never binds	$\Theta(V)$

Packing is polynomial. Backtracking is exponential on that family, which is why the experiment has a time cap. Extra space beyond the graph is linear in V.

Functions. `loadCatalog` reads rows and rejects a bad credit, term, or unknown clause member. `loadStudent` reads

one synthetic student. `buildIndex` / `searchNode` maintain the BST; a duplicate code is an error. `buildGraph` fills counts and adjacency lists. `topologicalOrder` returns false if a required course never becomes ready. `greedyPlan` packs along that order. `backtrackPlan` is Algorithm 3. `unlocked` applies the concurrent rule. `verifyPlan` rechecks prerequisites, offerings, and the cap. `scorePlan` returns span, then preference. `insertionSort` orders a ready list. `runTimingSeries` takes five trials and the median, and stops at a time cap. `showMenu`, `savePlan`, and `freeTree` do what the names say. Planners do not read `cin`. Tests call them directly.

Pseudocode, written before the C++.

Algorithm 1 Main program

Input: catalog path, and a student path when the user loads one
Output: both plans printed, or the bad row named

```
courses, clauses ← loadCatalog(path)
if the load failed, or a code is duplicated then
    print the row and stop
end if
buildGraph(courses, clauses)
repeat
    choice ← showMenu
    if choice is outside 1–9 then
        print "Enter a number from 1 to 9"
    else if choice is 4, 5, or 6 then
        if no student is loaded then
            print "Load a student first"
        else
            run the requested planners, verifyPlan(result), print span, preference, and time
        end if
    else if choice is 9 then
        freeTree(root) and stop
    end if
until stopped
```

Algorithm 2 Earliest-semester packing

Input: courses, clauses, completed set, cap, and S semesters
Output: a plan, or failure if the order cannot be built

```
order ← topologicalOrder(courses, completed)
if order is incomplete then
    return failure
end if
load[1 . . . S] ← 0
for each not-yet-completed course c along order do
    placed ← false
    for s ← 1 to S do
        if c is offered in s, the cap has room, and some clause is satisfied (strict members earlier; concurrent members in s or earlier) then
            put c in s, add its credits to load[s], set placed, and break
        end if
    end for
    if placed is false then
        return failure
    end if
end for
return the plan
```

Algorithm 3 Backtracking plan

Input: the topological order, the cap, and S
Output: the stored plan with the smallest span

```
1: best.span ← S + 1;   SEARCH(1);   return best

2: function SEARCH(i)
3:     if i is past the end of order then
4:         if this span is smaller than best.span then
5:             best ← this plan
6:         end if
7:         return
8:     end if
9:     c ← order[i]
10:    if c is already completed then
11:        SEARCH(i + 1);   return
12:    end if
13:    for s ← 1 to S do
14:        if s cannot beat best.span then
15:            return
16:        end if
17:        if c is offered in s, the cap holds, and some clause is satisfied then
18:            place c in s;   SEARCH(i + 1);   undo
19:        end if
20:    end for
21: end function
```

Semesters are tried earliest first, so a later s cannot beat a span already found and the inner return is the prune. On a chain with one legal semester per course, the first plan is the only plan. That is the $\Theta(V + P)$ case.

Tests. T1–T2 use the published clauses. T3–T6 and T9 use a constructed file, not the catalog: P and Q are fall-only, R is spring-only, Q is a strict prerequisite of R, the cap is one course, and the calendar starts in a fall.

ID	Kind	Input	Expected
T1	Normal	Look up EECE 2560	Clause EECE 2160 or CS 1500. Credits 4.
T2	Normal	Completed set is EECE 2160	EECE 2560 and EECE 2322 unlock. EECE 2160 is not planned again.
T3	Normal	P, Q, R, cap 1, both planners	Both pass `verifyPlan`. R is never beside or before Q.
T4	Normal	Same file, compare spans	Greedy span 4: P, empty spring, Q, R. Backtracking span 3: Q, R, P.
T5	Boundary	Every required course already done	Both plans empty, span 0.
T6	Boundary	Cap 0, one course left	Both fail. No plan file is written.
T7	Invalid	Duplicate code EECE 2560	Load stops on that row.
T8	Invalid	Clause names EECE 9999	Load stops and names EECE 9999.
T9	Invalid	Add edge $R \to Q$	Order is incomplete. Neither planner prints a semester list.
T10	Invalid	Menu 0, 10, and a blank line	"Enter a number from 1 to 9."

Experiment. Worst-case family from the bound: $P = 0$, every course offered every term, cap loose, $S = 4$, $V \in \{8, 10, 12\}$, five trials, median time. A trial past the cap is recorded as capped. Packing should stay fast; backtracking's median should jump. No millisecond guess before measuring. A second run uses the transcribed BSCmpE file and three synthetic students (nothing completed, the first programming course completed, one preferred elective). Both plans must pass `verifyPlan`. Which span is smaller waits on that file.

4.2 GitHub setup

Repository `EECE7205_HW`, private, only collaborator, default branch `main`: `https://github.com/mjsuhhh/EECE7205_HW`. Homework stays where it is. Project files go under `FinalProject/`. Instructor read access if the course asks.

`main` stays compilable. Feature branches: `feature/load-catalog`, `feature/prereq-graph`, `feature/greedy-plan`, `feature/backtracking`. Each opens a pull request. I review my own diff before merging: builds with `g++ -std=c++17 -Wall -Wextra`, new tests pass, the function matches the pseudocode, and `verifyPlan` is not the planner it checks. Commits stay small (`Add the BSCmpE catalog CSV`, `Add clause-count topological order`). Order of work: catalog and README, then the graph, packing, backtracking, then the timing script.

`FinalProject/README.md` comes with the first code: description, both catalog URLs, CSV columns, OR and concurrent rules, build and run, and progress. Build and run: `g++ -std=c++17 -Wall -Wextra -o planner src/main.cpp` and `./planner data/catalog.csv`. Progress now: topic, sources, pseudocode, bounds, and tests. No `.cpp` yet.

5 Tools and Programming Language

No external libraries. The graph, both planners, the queue, the BST, and insertion sort are mine. Files use `<fstream>` and `<sstream>`. Timing uses `<chrono>`. A CSV or graph library would hide the clauses and the search.

Tool	Use	Skill
C++17	Structs, functions, references, arrays, pointers. No class hierarchy.	Existing, on a larger program
g++ and clang	`-std=c++17 -Wall -Wextra`, as in the homework.	Existing
Visual Studio Code	Edit and read compiler diagnostics.	Existing
Git and GitHub	Branches, commits, and pull requests on `EECE7205_HW`.	Commits existing; pull requests new
GDB or the VS Code debugger	Stop in `unlocked` and `Search`.	To develop
Catalog and student CSV	Real clauses transcribed. Synthetic student rows.	To develop
T1–T10 and timing	Correctness, then $P = 0$ at $V = 8, 10, 12$.	Bounds existing; timing a search is new
`run_tests.sh`	Compile and run the non-menu checks, as in Assignment 01.	Existing pattern
GitHub Actions	Not used. The shell script is the check.	Not used